Developer Interview Sheet

GigResource · second developer lead · prepared 12 November 2026

How to use this. Every question below is about work this candidate would actually do on GigResource. You do not need to be technical to score it: under each question is what a good answer contains and what should worry you. Read the question as written, let them answer, then tick.

If you only have ten minutes, ask questions 3, 5, 7 and 8. The first three are the ones where a wrong answer costs us data, money or a client's trust, and the fourth shows how they behave when something is already broken.

About the work itself

1. GigResource is built with Laravel and Blade templates, with MySQL behind it. There is no React or Vue anywhere. Have you built a complete site this way, and are you happy to keep working in it?

1 of 12

YES Names a specific project they built in Blade. Says plainly that they are comfortable.

NO Wants to rebuild the front end in React, Vue or Inertia. Says they only build APIs and someone else does the screens.

The site is over three hundred Blade files and three thousand tests. Nobody is rewriting the front end, so a candidate who only wants to work the other way will be unhappy within a week.

2. A page lists fifty events, and each one shows the client's name. The page runs fifty-one database queries. What is happening, and how would you fix it?

2 of 12

YES Calls it an N+1 problem. Says to load the related records together, usually with "eager loading" or the word *with()*.

NO Suggests adding caching, or a bigger server. Does not know the term. Goes quiet.

This is the single most common performance mistake in Laravel and it is on every list page we have. Anyone who has maintained a real Laravel site has met it.

3. What is the difference between *php artisan migrate* and *php artisan migrate:fresh*, and when would you use each? 3 of 12

YES Says plainly that *fresh* deletes everything and rebuilds, so it is for a local machine only, never on the live site. *migrate* applies what is new.

NO Treats them as interchangeable. Says they would run *fresh* to tidy things up.

One wrong command here deletes every client, booking and payment on the live site. The answer has a single correct shape, which makes it easy to score.

4. A column in the live database has the wrong name and we want to rename it. The table already holds real client data. How do you do it safely? 4 of 12

YES Writes a new migration. Keeps the old data, copies it across, then removes the old column. Says you never edit a migration that has already run.

NO Says to edit the original migration file, or to rebuild the database.

We have nearly two hundred of these change files and a live database behind them.

Money, which is the part that must not be wrong

5. How do you store a price in the database, and why would you not use a decimal number that floats? 5 of 12

YES Either a fixed decimal column, or whole numbers of cents. Must say that a floating-point number is wrong for money because it loses fractions of a cent.

NO Says float or double. Says it does not matter at our size.

Every agreement, deposit, fee and commission on this site is money. A rounding error here is an argument with a client.

6. The same booking total appears on four different pages of a client's account. How would you make sure all four always agree? 6 of 12

YES Works the figure out in one place and has all four pages use it. Mentions writing a test so it cannot drift apart again.

NO Says they would check each page, or copy the same query into each one. Has not thought about it.

This exact fault has already happened here: two pages said eighty dollars while two said five thousand and eighty. It is the difference between a developer who thinks in systems and one who thinks in screens.

7. A payment provider tells our site that a card charge succeeded. The same message arrives twice by mistake. What stops the client being charged twice?

7 of 12

YES Recognises this as a webhook. Says each message carries an identity that we record, and we ignore one we have already seen. May use the word idempotent.

NO Does not know what a webhook is. Says it would not happen.

Payment providers deliver the same message twice on purpose when they are unsure it arrived. Not handling it means double charges.

Working alone, and under pressure

8. A page works on your own machine and shows an error on the live site. Talk me through your first five minutes.

8 of 12

YES Reads the error log first. Reproduces the problem. Only then changes anything. Looks at what differs between the two machines.

NO Starts changing code straight away to see what helps. Says they would ask someone else to look.

On this project there is nobody else to ask at two in the morning. Someone who guesses instead of reading the log turns an hour into a day.

9. Have you ever been the only developer on a project, responsible for putting it live yourself? Tell me about that one.

9 of 12

YES A real example, named. Describes deploying it themselves and what went wrong at least once.

NO Has always had a team lead who handled releases. Cannot name a project.

This is the job. There is no senior developer above them to catch a mistake before it reaches the live site.

10. This project has about three thousand automated tests, and they are run before anything goes live. How do you feel about that?

10 of 12

YES Treats it as normal and useful. Says they run them before pushing. Understands that for one developer the tests are the only second pair of eyes.

NO Says tests slow you down, or that testing is somebody else's job.

A candidate who resents the tests will stop running them, and then we will find out from a client.

Judgement

11. We only operate in seven states, and a professional must be in the same state as the client. Where in the system would you put a rule like that so it cannot be got around?

11 of 12

YES On the server, in one place, checked every time. Says that hiding a button is not a rule.

NO Says they would hide the option on screen, or check it in the browser only.

Our geography rules are a legal position, not a convenience. A rule that only exists on screen is not a rule.

12. The owner asks for a feature you think is a mistake. What do you do? And if you notice a completely unrelated bug while you are in there?

12 of 12

YES Says the concern once, with reasons, then builds what the owner decides. Writes the unrelated bug down and raises it, rather than quietly fixing it inside unrelated work.

NO "Whatever they ask, I build." Or refuses outright. Or fixes everything they notice without telling anyone.

Both halves matter. The first is whether they will warn us. The second is whether a small change can be trusted to stay small.

Scoring

Count of good answers	What it means
10 or more	Strong. Can be trusted with the live site fairly quickly.
7 to 9	Worth a second conversation. Check which ones they missed: 3, 5 and 7 matter more than the rest.
Under 7	Not for this project. They would be learning on a site that already has clients on it.

Three that are not negotiable. If they get question 3, 5 or 7 wrong, the count does not matter. Those three are the ones where a wrong answer destroys data, money or trust.

Red flags, whatever the score

- Every answer is "yes, I can do that", with no detail and no example.
- Cannot name a single project they worked on, or what it did.
- Cannot say which version of Laravel they last used.
- Blames previous clients or employers for everything that went wrong.
- Will not say "I don't know" about anything.